

UNIVERSITY OF SCIENCE - VNUHCM

FACILITY OF INFORMATION TECHNOLOGY

ADVANCED PROGRAM FOR COMPUTER SCIENCE

Chess Game Report

Project: Chess Game

Subject: CS160

Team member:

Nguyen Van Le Bao (24125053)

Phan Binh Nguyen Lam (24125014)

Teacher:

MSc Truong Phuoc Loc

MSc Do Nguyen Kha

December 21, 2024



Contents

1	Requirements	1
2	Basic overview	1
2.1	Game logic	1
2.2	Game modes	1
2.3	User interface (UI)	2
2.4	Chess Board Representation	2
2.5	AI Opponent	3
2.6	Game State Management	3
2.7	Technology Stack	3
3	Implementation	4
3.1	Code structure	5
3.2	Code Organization	6
3.3	Libraries used:	6
3.4	Major Components of the Codebase	6
4	Extra Feature	7
4.1	Load and Save Game UI	7
4.2	Theme	7
4.3	Music	7
5	Testing	8
6	Limitation and future improvement	9
6.1	Limitation	9
6.2	Future improvement	9
7	Tools	9

1 Requirements

The majority of the project's libraries are either embedded within the code or developed by the team itself, with the exception of the QT6 library, which is utilized for the graphical user interface (GUI).

2 Basic overview

The C++ Chess Project is a software application designed to simulate a chess game. The project encompasses the core functionalities of a chess game, including piece movement, game rules enforcement, and a graphical user interface (GUI). Key features typically include:

2.1 Game logic

- Implementing standard chess rule including moves for all pieces, special moves such as castling, promotion, en passant.
- Automatic detection of check, checkmate, stalemate and draw and notification.
- Only valid moves are allowed.

2.2 Game modes

Introduce two offline game modes:

- Player vs Player: mode where player can play against each other locally. Turn is manage automatically.
- Player vs Computer: where player can play against AI with 3 different level
 - Easy: the computer make random move.
 - Medium: the computer make a mixed of good and bad move.
 - Hard: the computer make the most logical move possible.

2.3 User interface (UI)

Implemented a graphical user interface (GUI) that includes:

- A chessboard where pieces can be move by clicking or dragging.
 - Turn indicator for white or black.
 - Notification and sound for making move, check, checkmate, stalemate or draw.
 - Move history panel.
 - Include buttons or options to:
 - Start a new game.
 - Exit game.
 - Choose between starting color, game mode and difficulty (for AI game mode).
 - Undo/redo move.
 - Save/load game.
 - Customize UI:
 - * Board color.
 - * Pieces style.
 - * Sound effects.
 - * Background music.
- ...

2.4 Chess Board Representation

- Implement a graphical chessboard using a C++ GUI library Qt.
- Highlight legal moves when a player selects a piece.
- In 2-player mode, alternate between Player 1 (white) and Player 2 (black) automatically after each move.
- In Player vs AI mode, alternate between the player's move and the AI's move. The AI make a move immediately after the player completes theirs.

2.5 AI Opponent

In our project, we utilize stockfish.exe to integrate advanced chess engine capabilities for AI decision-making. Stockfish provides the best possible moves based on the current game state by analyzing positions with high computational accuracy.

- Configuration and Modes:
 - Depth Configuration:
 - * We control the engine's search depth to balance performance and decision making complexity. Lower depths yield faster but less optimal moves, while higher depths increase decision accuracy.
 - Skill Level Adjustment:
 - * Stockfish allows skill levels to be configured from 0 (beginner) to 20 (maximum skill). We leverage this feature to create three difficulty modes:
 - Easy: Low depth and reduced skill level for simpler moves.
 - Medium: Moderate depth and skill level for balanced gameplay.
 - Hard: High depth and maximum skill level for challenging gameplay.

2.6 Game State Management

- Maintain the current board and game state.
- Manage turns between players or AI.
- Implement **Undo and Redo** features:
 - Use a history stack to store moves for undo operations.
 - Use a second stack to handle redo operations.
- Allow resetting the game to its initial state at any time.

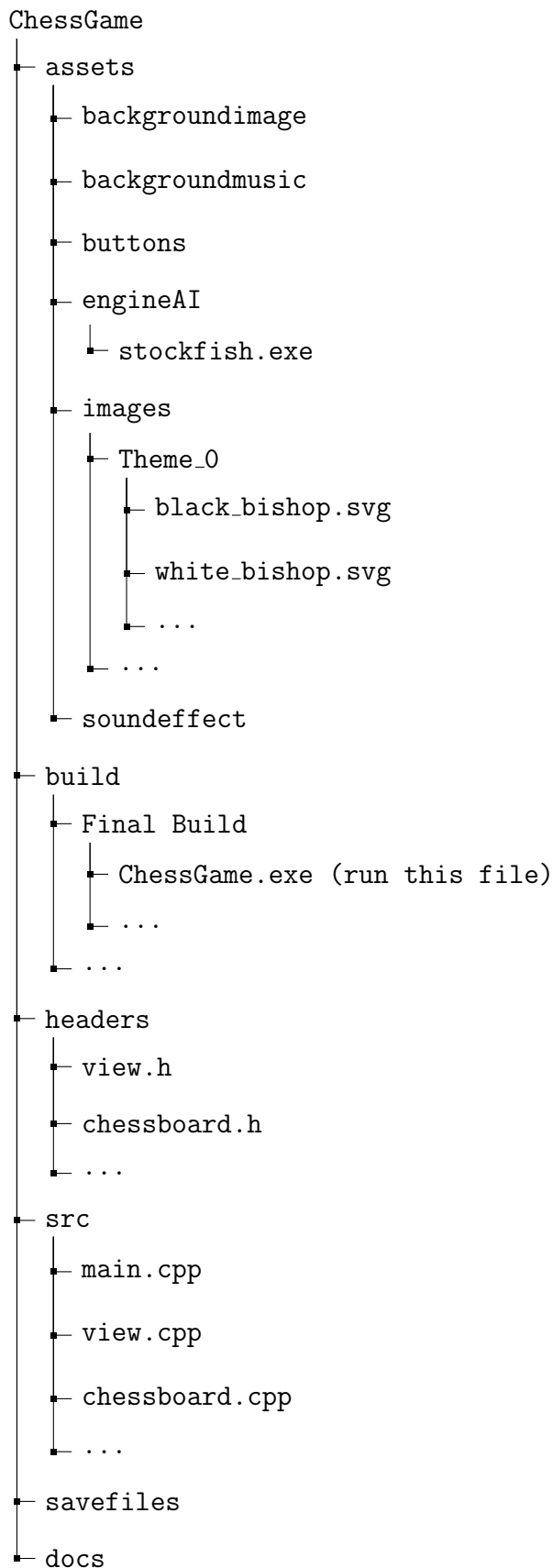
2.7 Technology Stack

- **C++ for Core Logic:** Write all game logic and game state management in C++.

- **AI Integration:** Use `stockfish.exe` for AI decision-making in medium and hard difficulty levels.
- **GUI Library:** Use Qt to build the user interface, render the chessboard, and manage input events.
- **Save Game:** Serialize the current game state, including piece positions, turn information, and other necessary data, and save it in a JSON file.
- **Load Game:** Open a file dialog to select a saved game file, deserialize the JSON data, and restore the game state, updating the chessboard and relevant UI elements.

3 Implementation

3.1 Code structure



3.2 Code Organization

- **src/**: Contains the source code for the application.
- **header/**: Includes external libraries required by the application.
- **assets/**: Stores media files such as sound, music, fonts, and chess piece images.
- **build/**: Holds files generated during the compilation and build process.
- **savefiles/**: The folder is used to store serialized game states. This allows players to save their progress at any point and restart the game later.

3.3 Libraries used:

- Standar C++ library.
- Qt for graphics.
- We use **chess.hpp** to handle logic, chessboard representation, and generation of legal moves, serving as the core of the game's mechanics.

3.4 Major Components of the Codebase

- **Single player mode**: We input the FEN board representation into **stockfish.exe**, which generates the best move. The bot executes this move, after which it is the player's turn. The player moves and the FEN board is updated and sent back to **stockfish.exe**. This process alternates, with the player and the bot taking turns to make their moves.
- **Two Player Mode**: Two players take turns making moves until the game ends.
- **Game State Management Using Scenes**: We manage the game's states by implementing a scene-based system, similar to transitioning scenes in a movie. All scenes are interconnected through a main controller called **view.h**, which acts like a camera. Each scene contains buttons that notify the main controller to transition to the next scene.

- The game begins with the **main menu scene**, which includes buttons like *Play Game*, *Load Game*, *Themes*, and *Settings*. These buttons transition to their respective scenes: `chessboard`, `loadgameui`, `themesetting`, or `settingsmenu`.
- Within each scene, buttons are available to allow transitions to other scenes as needed.

Inside the **main menu**, there are sub-states that operate like elements in an array. These states are pre-initialized and numbered sequentially (e.g., 0, 1, 2, ...). Lower-numbered states are displayed first. Each state specifies the buttons to appear for interaction. When a button is pressed, it sends a signal to the active scene, causing it to update the current state—either progressing to the next state or reverting to the previous one.

In essence, all scenes are pre-drawn, and `view.h` determines which scene is displayed based on player interaction. Each scene, when active, is rendered with its corresponding states and transitions.

4 Extra Feature

4.1 Load and Save Game UI

- Save Game: We allow users to name their saved game files, with no limitations on the number of files that can be saved.
- Load Game: The game load interface displays a preview of the game board and allows users to delete or restart previously saved games.

4.2 Theme

- We provide a preview feature for the available themes in the game, allowing players to conveniently choose their preferred theme.

4.3 Music

- Players can upload their favorite music tracks (only files in **.mp3** format are supported) by placing them in the `assets/backgroundmusic` folder.

- After restarting the game, the new music will appear in the game's settings menu for selection.

5 Testing

To ensure the functionality and reliability of the C++ Chess Project, a comprehensive testing methodology is employed:

- Unit testing is done individually for each library to validate the core components of the game, such as individual piece movement, rule enforcement and special moves like casting or en passant.
- Automated test cases are implemented to systematically evaluate edge case, including invalid moves and complex game scenarios. Test cases are fetched from 1000 real chess games from chess.com.
- Integration testing verifies the interaction between components, such as the board, pieces, and game logic. For the GUI, user interface testing ensures that the board and pieces are displayed correctly, user inputs are processed accurately, and the overall interface is intuitive.
- Real-world testing is conducted with users to simulate real-world usage, identify bugs and assess the player experience.

The intense testing shows that:

- Each library and component is correctly implemented to handle every aspect of the game.
- The game passes all 1000 test cases successfully with no unexpected behaviour.
- Integration testing shows that the interaction between components is smooth and accurate, ensuring the best user experience.
- Below is the comment from my roommate's use for real-world testing:
 - "This game is so hilarious, the music the sound and the pieces look so funny. I love this game."
 - "You should have a 10 for your project. This game is magnificent, I encounter no crash or nothing during my playing. The AI bot is very good too".

- "Fun and interactive gameplay. 10 out of 10."

The C++ Chess Project successfully delivers a functional and engaging chess application with accurate game mechanics, a user-friendly interface, and seamless gameplay. The integration of the QT6 library enhances the GUI, while rigorous testing ensures reliability and performance. Feedback from playtesting further refined the user experience, resulting in a polished and robust platform.

6 Limitation and future improvement

6.1 Limitation

The C++ Chess Project has some limitations that could be addressed in future improvements. The project lacks online multiplayer functionality, restricting gameplay to local users. Additionally, the graphical interface, while functional, could be further enhanced with more advanced visual effects and animations. Lastly, while the testing was extensive, unexpected edge cases can arise during extended use, requiring additional updates and refinements.

6.2 Future improvement

- Multiplayer and online mode with elo system and ranking.
- More advanced visual effects and animations.
- More game modes and variants such as 4 players mode, 960Chess, etc.
- Game timer with multiple modes and custom timer with timeout.
- Watching mode and real-time/after game analysis using AI.

7 Tools

To facilitate effective teamwork during the development of the C++ Chess Project, several collaboration tools were utilized:

- Github git for code sharing and version control

- Messenger, Discord for sending message, meeting.
- Ultraview for debugging local error.
- Visual Studio Live Share for collaborate coding.